Project_Methods

Nathan Hogg

2026-04-15

Define Data

Adam et al. "Clustering and superspreading potential of SARS-C-oV-2 infections in Hong Kong". In:Nature Medicine(2020).doi:https://doi.org/10.1038/s41591-020-1092-0.

```
pairs <- read.csv("transmission_pairs.csv")

infector_counts <- table(pairs$infector.case)

all_infectees <- unique(pairs$infectee.case)
all_infectors <- unique(pairs$infector.case)
terminal_cases <- setdiff(all_infectees, all_infectors)

secondary_cases <- c(as.integer(infector_counts), rep(0, length(terminal_cases)))

freq_table <- as.data.frame(table(secondary_cases))
colnames(freq_table) <- c("secondary_cases", "frequency")

print(freq_table)
```

```
##   secondary_cases frequency
## 1               0       153
## 2               1        57
## 3               2        18
## 4               3         6
## 5               4         5
## 6               5         1
## 7               6         2
## 8              10         1
## 9              11         1
```

```
freq_table$secondary_cases <- as.integer(as.character(freq_table$secondary_cases))
raw_data <- rep(freq_table$secondary_cases, freq_table$frequency)
```

ZIP Distributions

Application of Zero Inflated Poisson (ZIP) distribution to model the number of secondary cases per infector, accounting for excess zeros in the data. ## Results Lambda (Poisson Mean): 1.398481 Sigma (Zero-Inflation Probability): 0.5047316 Log-Likelihood: -295.6031 AIC: 595.2061 Overall mean (lambda * (1 - sigma)): 0.6926233 ## Interpretation The estimated Poisson Mean (Lambda) of approximately 1.398 suggests that, on average, an infector is expected to cause about 1.398 secondary cases in the absence of zero-inflation. The Zero-Inflation Probability (Sigma) of approximately 0.505 indicates that about 50.5% of cases are

"structural zeros", meaning they are not expected to cause any secondary cases due to factors such as isolation, hospitalization, or other interventions. The overall mean number serves as the effective reproduction number (R), which matches with Adam et al.'s estimate of R = 0.69. This suggests that, on average, each infector is expected to cause less than one secondary case, indicating that the outbreak may be under control in this population. The Akaike Information Criterium (AIC) will be used to compare the fit of the ZIP model to other models to determine which one best captures the distribution of secondary cases in the data. Finally, we can check the prediction of the model by evaluating the predicted zeros against the observed zeros in the data:

$$P(X = 0) = \sigma + (1 - \sigma) \cdot e^{-\lambda} = 0.505 + 0.495 \cdot e^{-1.398} \approx 0.505 + 0.495 \cdot 0.247 = 0.505 + 0.122 = 0.627 \quad (1)$$

As seen here, the model's prediction was correct within the observed data, which had a zero proportion of approximately 0.63. This suggests that the ZIP model is a good fit for the data, capturing the excess zeros effectively.

```r
# Log-Likelihood Function
zip_loglik <- function(params, data) {
  lambda <- params[1] # Poisson mean
  sigma <- params[2]  # Zero-inflation probability

  # Constraints: lambda > 0, 0 < sigma < 1
  if (lambda <= 0 || sigma <= 0 || sigma >= 1) return(-Inf)

  zeros <- data[data == 0]
  nonzeros <- data[data > 0]

  # Log-Likelihood for zeros: log(sigma + (1 - sigma) * exp(-lambda))
  ll_zeros <- length(zeros) * log(sigma + (1 - sigma) * exp(-lambda))

  # Log-Likelihood for non-zeros: log(1 - sigma) + log(Poisson probability)
  ll_nonzeros <- sum(log(1 - sigma) + dpois(nonzeros, lambda, log=TRUE))

  return(ll_zeros + ll_nonzeros)
}

# Negative log-likelihood for optim
zip_negloglik <- function(params, data) {
  return(-zip_loglik(params, data))
}

# Fit the model
fit_zip <- function(data) {
  # Starting values
  p_zero <- mean(data == 0)
  start_sigma <- max(0.01, min(0.99, p_zero - 0.1))
  start_lambda <- mean(data) / (1 - start_sigma)

  result <- optim(
    par = c(start_lambda, start_sigma),
    fn = zip_negloglik,
    data = data,
    method = "L-BFGS-B",
    lower = c(1e-6, 1e-6),
```

```r
    upper = c(Inf, 1 - 1e-6),
    hessian = TRUE
  )

  lambda_hat <- result$par[1]
  sigma_hat <- result$par[2]
  loglik <- -result$value
  aic <- 2 * 2 - 2 * loglik # 2 params

  cat("Lambda (Poisson Mean):", lambda_hat, "\n")
  cat("Sigma (Zero-Inflation Probability):", sigma_hat, "\n")
  cat("Log-Likelihood:", loglik, "\n")
  cat("AIC:", aic, "\n")
  cat("Overall mean (lambda * (1 - sigma)):", lambda_hat * (1 - sigma_hat), "\n")

  return(list(lambda = lambda_hat, sigma = sigma_hat, loglik = loglik, aic = aic, fit = result))
}

# Run on raw data
zip_result <- fit_zip(raw_data)
```

```
## Lambda (Poisson Mean): 1.398481
## Sigma (Zero-Inflation Probability): 0.5047316
## Log-Likelihood: -295.6031
## AIC: 595.2061
## Overall mean (lambda * (1 - sigma)): 0.6926233
```

```r
# Check predicted vs. observed zeros
n <- length(raw_data)
observed_zero_prop <- mean(raw_data == 0)

lambda <- zip_result$lambda
sigma <- zip_result$sigma

predicted_zero_prop <- sigma + (1 - sigma) * exp(-lambda)

cat("lambda:", lambda, "\n")
```

```
## lambda: 1.398481
```

```r
cat("sigma:", sigma, "\n")
```

```
## sigma: 0.5047316
```

```r
cat("Observed Zero Proportion:", observed_zero_prop, "\n")
```

```
## Observed Zero Proportion: 0.6270492
```

```r
cat("Predicted Zero Proportion:", predicted_zero_prop, "\n")
```

```
## Predicted Zero Proportion: 0.627049
```

```r
cat("Observed zero count:", sum(raw_data == 0), "out of", n, "\n")
```

```
## Observed zero count: 153 out of 244
```

```r
cat("Predicted zero count:", round(predicted_zero_prop * n), "out of", n, "\n")
```

```
## Predicted zero count: 153 out of 244
```

Beta-Poisson Mixture Modeling

```r
# Beta-Poisson Mixture Model
bp_pmf <- function(x, lambda, phi, nu) {
  N <- 1 / nu
  alpha1 <- lambda * phi
  alpha2 <- (N - lambda) * phi

  if (alpha1 <= 0 || alpha2 <= 0 || N <= lambda) return(rep(-Inf, length(x)))

  sapply(x, function(k) {
    f <- function(p) dpois(k, lambda = p * N) * dbeta(p, alpha1, alpha2)
    val <- tryCatch(integrate(f, 0, 1, rel.tol = 1e-8)$value,
                    error = function(e) 0)
    if (val <= 0) return(-Inf)
    log(val)
  })
}

bp_negloglik <- function(params, data, lambda) {
  phi <- params[1]
  nu <- params[2]
  if (phi <= 0 || nu <= 0 || nu >= 1/lambda) return(1e10)
  ll <- sum(bp_pmf(data, lambda, phi, nu))
  if (is.na(ll) || is.infinite(ll)) return(1e10)
  return(-ll)
}

fit_bp <- function(data) {
  lambda <- mean(data)
  best <- list(value = 1e10)

  for (sp in c(0.1, 0.5, 1, 5)) {
    for (sn in c(0.01, 0.1, 0.5)) {
      sv <- sn / lambda
      if (sv >= 1/lambda) next

      res <- tryCatch(
        optim(c(sp, sv), bp_negloglik, data = data, lambda = lambda,
              method = "L-BFGS-B",
              lower = c(1e-6, 1e-8),
              upper = c(100, 1/lambda - 1e-6)),
        error = function(e) list(value = 1e10))

      if (res$value < best$value) best <- res
    }
  }

  phi_hat <- best$par[1]
  nu_hat <- best$par[2]
  N_hat <- 1 / nu_hat
```

```r
  loglik <- -best$value
  aic <- 6 - 2 * loglik

  cat("Beta-Poisson MLE Results\n")
  cat("Lambda (mean):", lambda, "\n")
  cat("Phi (overdispersion):", phi_hat, "\n")
  cat("Nu (inverse contacts):", nu_hat, "\n")
  cat("N (mean contacts):", N_hat, "\n")
  cat("Log-likelihood:", loglik, "\n")
  cat("AIC:", aic, "\n")

  # Check if converged to negative binomial limit
  if (nu_hat < 0.01) {
    cat("\nNote: Nu ~ 0 (N → infty), beta-Poisson has converged to its\n")
    cat("negative binomial limit with theta =", 1/phi_hat, "\n")
  }

  return(list(lambda = lambda, phi = phi_hat, nu = nu_hat,
              N = N_hat, loglik = loglik, aic = aic))
}

bp_result <- fit_bp(raw_data)
```

```
## Beta-Poisson MLE Results
## Lambda (mean): 0.692623
## Phi (overdispersion): 0.804689
## Nu (inverse contacts): 0.0004845876
## N (mean contacts): 2063.61
## Log-likelihood: -276.2956
## AIC: 558.5912
##
## Note: Nu ~ 0 (N → infty), beta-Poisson has converged to its
## negative binomial limit with theta = 1.242716
```

MLE

```r
# Negative Binomial MLE
lambda_nb <- mean(raw_data)

nb_negloglik <- function(theta, data) {
  -sum(dnbinom(data, size = theta, mu = mean(data), log = TRUE))
}

nb_fit <- optim(par = 1, fn = nb_negloglik, data = raw_data, method = "L-BFGS-B", lower = 1e-6)

theta_hat <- nb_fit$par
loglik_nb <- -nb_fit$value
aic_nb <- 2 * 2 - 2 * loglik_nb
cat("Negative Binomial MLE Results\n")
```

```
## Negative Binomial MLE Results
```

```r
cat("Lambda (mean):", lambda_nb, "\n")
```

```
## Lambda (mean): 0.692623
```

```r
cat("Theta (overdispersion):", theta_hat, "\n")
```

```
## Theta (overdispersion): 0.5576684
```

```r
cat("Log-likelihood:", loglik_nb, "\n")
```

```
## Log-likelihood: -276.2905
```

```r
cat("AIC:", aic_nb, "\n")
```

```
## AIC: 556.581
```

```r
# Poisson MLE
lambda_poi <- mean(raw_data)
loglik_poi <- sum(dpois(raw_data, lambda_poi, log = TRUE))
aic_poi <- 2 * 1 - 2 * loglik_poi

cat("\nPoisson MLE Results\n")
```

```
##
## Poisson MLE Results
```

```r
cat("Lambda (mean):", lambda_poi, "\n")
```

```
## Lambda (mean): 0.692623
```

```r
cat("Log-likelihood:", loglik_poi, "\n")
```

```
## Log-likelihood: -320.7387
```

```r
cat("AIC:", aic_poi, "\n")
```

```
## AIC: 643.4775
```

```r
# Geometric MLE
lambda_geo <- mean(raw_data)
loglik_geo <- sum(dgeom(raw_data, prob = 1/(1 + lambda_geo), log = TRUE))
aic_geo <- 2 * 1 - 2 * loglik_geo

cat("\nGeometric MLE Results\n")
```

```
##
## Geometric MLE Results
```

```r
cat("Lambda (mean):", lambda_geo, "\n")
```

```
## Lambda (mean): 0.692623
```

```r
cat("Log-likelihood:", loglik_geo, "\n")
```

```
## Log-likelihood: -279.4219
```

```r
cat("AIC:", aic_geo, "\n")
```

```
## AIC: 560.8439
```

```r
# Model comparison table
cat("\n AIC Comparison \n")
```

```
##
##  AIC Comparison
```

```r
comparison <- data.frame(
  Model = c("Poisson", "Geometric", "Neg Binomial", "ZIP", "Beta-Poisson"),
  Parameters = c(1, 1, 2, 2, 3),
  LogLik = c(loglik_poi, loglik_geo, loglik_nb,
             zip_result$loglik, bp_result$loglik),
  AIC = c(aic_poi, aic_geo, aic_nb,
          zip_result$aic, bp_result$aic)
)
comparison <- comparison[order(comparison$AIC), ]
print(comparison, row.names = FALSE)
```

```
##         Model Parameters    LogLik      AIC
##  Neg Binomial          2 -276.2905 556.5810
##  Beta-Poisson          3 -276.2956 558.5912
##     Geometric          1 -279.4219 560.8439
##           ZIP          2 -295.6031 595.2061
##       Poisson          1 -320.7387 643.4775
```

Maximum Likelihood Estimation

Branching Process

```r
# Branching Process Simulation

# Simulate one outbreak using a given offspring distribution
simulate_outbreak <- function(rfunc, n_gen = 20, n_index = 1) {
  cases_per_gen <- numeric(n_gen)
  cases_per_gen[1] <- n_index
  total_cases <- n_index

  for (g in 2:n_gen) {
    if (cases_per_gen[g - 1] == 0) break
    new_cases <- sum(rfunc(cases_per_gen[g - 1]))
    cases_per_gen[g] <- new_cases
    total_cases <- total_cases + new_cases
  }
  return(list(total = total_cases, by_gen = cases_per_gen))
}

# Random draw functions for each fitted model
r_zip <- function(n) {
  lambda <- zip_result$lambda
  sigma <- zip_result$sigma
  is_zero <- rbinom(n, 1, sigma)
  ifelse(is_zero, 0, rpois(n, lambda))
}

r_betapois <- function(n) {
  lambda <- bp_result$lambda
  phi <- bp_result$phi
```

```r
  nu <- bp_result$nu
  N <- 1 / nu
  alpha1 <- lambda * phi
  alpha2 <- (N - lambda) * phi
  p <- rbeta(n, alpha1, alpha2)
  rpois(n, p * N)
}

n_sims <- 10000
n_gen <- 20

zip_sizes <- numeric(n_sims)
bp_sizes <- numeric(n_sims)

set.seed(42)

for (i in 1:n_sims) {
  zip_sizes[i] <- simulate_outbreak(r_zip, n_gen)$total
  bp_sizes[i] <- simulate_outbreak(r_betapois, n_gen)$total
}

# Summary statistics
cat("Branching Process Results (", n_sims, "simulations)\n\n")
```

```
## Branching Process Results ( 10000 simulations)
```

```r
cat("--- ZIP ---\n")
```

```
## --- ZIP ---
```

```r
cat("Mean outbreak size:", mean(zip_sizes), "\n")
```

```
## Mean outbreak size: 3.2154
```

```r
cat("Median outbreak size:", median(zip_sizes), "\n")
```

```
## Median outbreak size: 1
```

```r
cat("Max outbreak size:", max(zip_sizes), "\n")
```

```
## Max outbreak size: 129
```

```r
cat("P(extinction by gen", n_gen, "):", mean(zip_sizes < 50), "\n")
```

```
## P(extinction by gen 20 ): 0.9967
```

```r
cat("P(outbreak > 10 cases):", mean(zip_sizes > 10), "\n\n")
```

```
## P(outbreak > 10 cases): 0.0643
```

```r
cat("--- Beta-Poisson ---\n")
```

```
## --- Beta-Poisson ---
```

```r
cat("Mean outbreak size:", mean(bp_sizes), "\n")
```

```
## Mean outbreak size: 3.2581
```

```r
cat("Median outbreak size:", median(bp_sizes), "\n")
```

```
## Median outbreak size: 1
```

```r
cat("Max outbreak size:", max(bp_sizes), "\n")
```

```
## Max outbreak size: 226
```

```r
cat("P(extinction by gen", n_gen, "):", mean(bp_sizes < 50), "\n")
```

```
## P(extinction by gen 20 ): 0.9955
```

```r
cat("P(outbreak > 10 cases):", mean(bp_sizes > 10), "\n\n")
```

```
## P(outbreak > 10 cases): 0.0609
```

```r
# Superspreading analysis (Lloyd-Smith et al. definition)
# Superspreader = generates cases above 99th percentile of Poisson(R)
R <- mean(raw_data)
ss_threshold <- qpois(0.99, R)
cat("=== Superspreading Analysis ===\n")
```

```
## === Superspreading Analysis ===
```

```r
cat("R (mean):", R, "\n")
```

```
## R (mean): 0.692623
```

```r
cat("Superspreading threshold (99th %ile of Poi(R)):", ss_threshold, "\n")
```

```
## Superspreading threshold (99th %ile of Poi(R)): 3
```

```r
cat("Observed superspreaders:", sum(raw_data > ss_threshold), "/", length(raw_data), "\n")
```

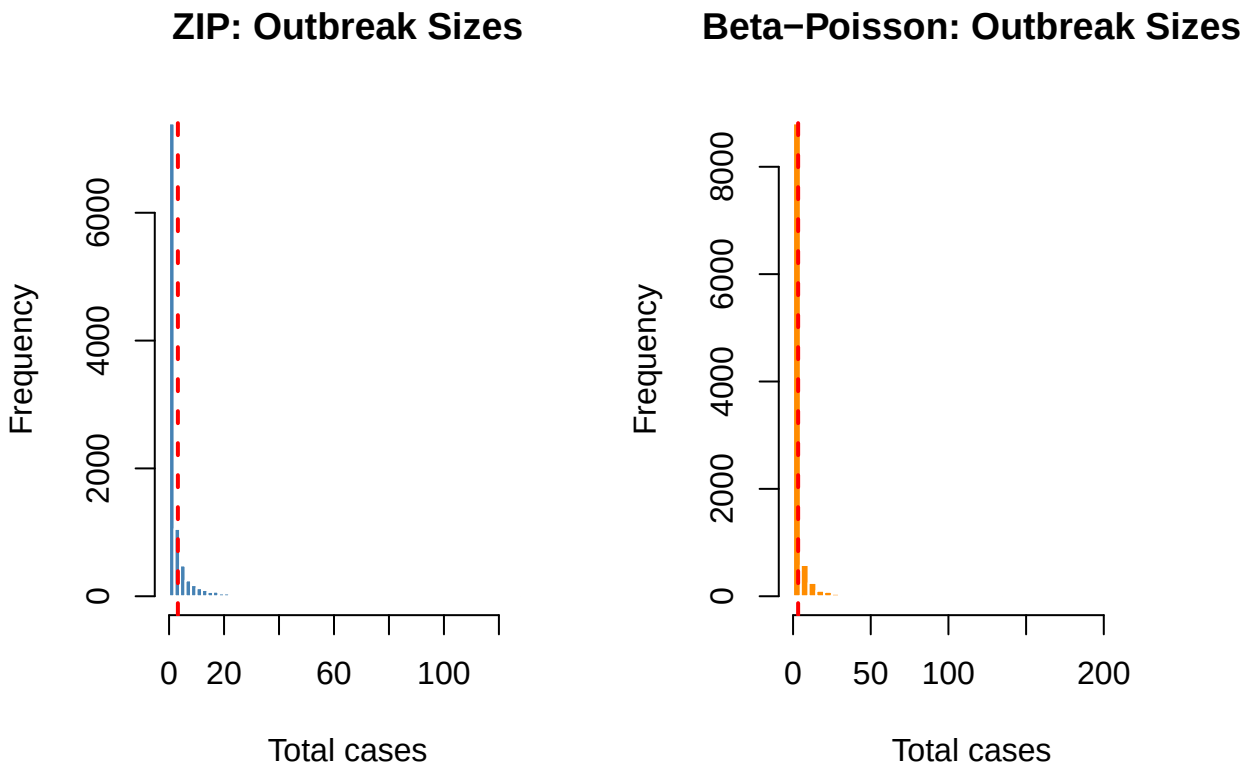
```
## Observed superspreaders: 10 / 244
```

```r
cat("Observed superspreader proportion:", mean(raw_data > ss_threshold), "\n")
```

```
## Observed superspreader proportion: 0.04098361
```

```r
# Plot outbreak size distributions
par(mfrow = c(1, 2))
hist(zip_sizes, breaks = 50, main = "ZIP: Outbreak Sizes",
     xlab = "Total cases", col = "steelblue", border = "white")
abline(v = mean(zip_sizes), col = "red", lwd = 2, lty = 2)

hist(bp_sizes, breaks = 50, main = "Beta-Poisson: Outbreak Sizes",
     xlab = "Total cases", col = "darkorange", border = "white")
abline(v = mean(bp_sizes), col = "red", lwd = 2, lty = 2)
```


ZIP: Outbreak Sizes
Frequency
0
2000
4000
6000
0
20
60
100
Total cases
Beta−Poisson: Outbreak Sizes
Frequency
0
2000
4000
6000
8000
0
50
100
200
Total cases